IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

ETAPA B · CONSTRUIR · SPRINT 5 DE 17

Sprint 5 · Matemáticas y estadística en código

¿podemos demostrar con código lo que vimos en teoría?

FECHAS
30/10 y 3/11

HORAS
12 h (3 h teoría · 6 h práctica, incluye 1 h de CRUD · 3 h PBL)

Qué aprenderás

1. A manejar **datos numéricos con NumPy**: arrays, vectorización, *broadcasting* y álgebra lineal.
2. A **entrenar un modelo desde cero**: entender el gradiente y programar la regresión lineal con descenso del gradiente.
3. A hacer **estadística con Python**: describir datos, usar distribuciones, simular (Monte Carlo) y construir intervalos de confianza.
4. A **contrastar hipótesis** e interpretar p-valores, tamaños de efecto y la diferencia entre **correlación y causalidad**.
5. A hacer **CRUD desde Python** sobre SQLite y MongoDB y a dejar una base de datos poblada.

Requisitos previos

- Haber hecho el Sprint 4 (Python básico, funciones, ficheros).
- Colab o Jupyter con Python 3.11. Los cuadernos usan `numpy`, `pandas`, `matplotlib`, `scipy`, `statsmodels`, `scikit-learn`, `sqlite3` y `mongomock`.
- Los datos (`ocupacion_diaria.csv`, `reservas_turisdata.csv`, `turisdata.db`) se descargan solos. Son **sintéticos**: sirven para practicar, no son estadísticas reales del turismo canario.

Prácticas (5 h)

Cada ejercicio se autocorrige con un `assert`: si falla, lee el mensaje, corrige y ejecuta de nuevo. Sigue el orden.

Cuaderno	Tema	Tiempo	Colab
`S05_01_numpy`	NumPy: arrays, máscaras, vectorización, broadcasting, semillas	40 min	Abrir en Colab
`S05_02_algebra_lineal`	Vectores, matrices, producto matricial, sistemas de ecuaciones, ecuación normal	40 min	Abrir en Colab
`S05_03_gradiente_regresion`	Derivada, MSE, descenso del gradiente y regresión lineal desde cero	85 min	Abrir en Colab
`S05_04_estadistica_descriptiva_ic`	Estadística descriptiva, distribuciones, Monte Carlo e intervalos de confianza	50 min	Abrir en Colab
`S05_05_contrastes_causalidad`	Contrastes de hipótesis, correlación frente a causalidad, `statsmodels`, Bayes	45 min	Abrir en Colab
`S05_06_crud_python`	CRUD desde Python: `sqlite3`, `pymongo` / `mongomock` (M6b)	40 min	Abrir en Colab

RETO Reto PBL · «¿La ocupación depende de esto?» (unas 3 h 20 min)

En pocas palabras (léelo primero)

Qué os pide el cliente. Al dueño de TurisData le han dicho dos cosas: «*cuando hace calor se llena menos*» y «*los festivos llenan*». Antes de cambiar personal y precios, quiere saber **cuánto** cambia realmente la ocupación diaria con la temperatura y con los festivos, y **con qué seguridad** se puede afirmar. Os da tres años de datos diarios.

Es el reto más «de números» hasta ahora, pero no hace falta ser matemático: la plantilla del cuaderno os guía y cada paso tiene una comprobación automática.

Qué vais a hacer, en seis pasos:

1. **Guardar los datos en una base de datos** (SQLite) y leerlos desde Python con SQL.
2. **Mirar los datos** con gráficos y medias por mes (¿qué se ve?).
3. **Programar vosotros una regresión lineal «desde cero»** con descenso del gradiente (sin la función ya hecha de una librería).
4. **Comprobarla** comparando con una librería (`statsmodels`): si coinciden, vuestra implementación es correcta.
5. **Calcular la seguridad de lo que afirmáis**: p-valores, intervalos de confianza y tamaño del efecto.
6. **Escribir una respuesta de máximo 150 palabras** para el cliente.

Qué vais a entregar: un cuaderno, una base de datos y la respuesta al cliente.

Qué vais a aprender haciéndolo: cómo «aprende» un modelo (ajustando números poco a poco), cómo medir **cuánta confianza** hay en un resultado y por qué **correlación no es causalidad**.

Cuánto tiempo: unas 3 horas y 20 minutos entre el planning, el reto y la entrega (30/10 y 3/11). El paso 1 (cargar los datos en una base) coincide con la práctica de CRUD.

Palabras que van a aparecer en este reto

Palabra	Qué significa, en sencillo
Regresión lineal	Dibujar la mejor recta que describe cómo cambia una cosa (ocupación) cuando cambia otra (temperatura)
Descenso del gradiente	Método para encontrar la mejor recta ajustando sus números poco a poco, «bajando una cuesta» de error hasta el fondo
Tasa de aprendizaje	El tamaño de cada «paso» al bajar la cuesta. Muy grande, se pasa de largo; muy pequeño, tarda siglos
Curva de aprendizaje	Gráfico del error en cada vuelta del algoritmo. Si baja y se estabiliza, funciona
Estacionalidad	Patrón que se repite cada año (más ocupación en verano, por ejemplo). Hay que **controlarla** para no confundirla con el efecto de la temperatura
Intervalo de confianza	Rango donde probablemente está el valor real. «El efecto es de −2 a −5 puntos, con un 95 % de confianza»
p-valor	Probabilidad de ver un resultado así **si en realidad no hubiera efecto**. Si es muy pequeño (p. ej. < 0,05), el efecto probablemente es real
Tamaño del efecto	**Cuánto** cambia la ocupación, no solo si cambia. Un efecto puede ser «real» pero minúsculo
Correlación y causalidad	Que dos cosas vayan juntas no prueba que una cause la otra
SQLite	Una base de datos sencilla, que vive en un único fichero
Semilla	Número que fija el azar para que los resultados se repitan

Contexto del cliente

TurisData Canarias quiere planificar personal y precios de su alojamiento de referencia. Le han dicho que «cuando hace calor se llena menos» y que «los festivos llenan». Antes de tomar decisiones quiere saber **cuánto** cambia la ocupación diaria con la temperatura y con los festivos, y **con cuánta seguridad** podemos decirlo. Tenéis su serie diaria de tres años (`ocupacion_diaria.csv`).

Qué tenéis que entregar

Un **cuaderno** (`S05_07_reto_ocupacion` , con la plantilla) y una **base de datos** con:

1. La serie diaria **cargada en una base SQLite** (`turisdata_reto.db`) y leída desde Python con SQL.
2. La **regresión lineal implementada desde cero** con descenso del gradiente (modelo simple y modelo con control de la estacionalidad).
3. La **comparación con una librería** (`statsmodels`).
4. El **análisis estadístico**: contraste, intervalos de confianza y p-valores.
5. Una **respuesta al cliente de máximo 150 palabras**: qué encontráis, qué recomendáis y qué no podéis afirmar.

Restricciones

- **Tiempo:** unas 3 h 20 min dentro del sprint (12 h), con el planning y la entrega.
- El descenso del gradiente lo escribís vosotros (sin `LinearRegression` en la implementación; la librería solo sirve para contrastar).
- Datos sintéticos: no generalicéis a otros alojamientos ni al turismo real.
- Semillas fijas y el cuaderno debe ejecutarse de arriba abajo sin errores.

Criterios de evaluación (rúbrica del curso, 0–4 en cada criterio)

Criterio	Qué se mirará en este reto
Corrección técnica	El gradiente converge y coincide con `statsmodels` ; los contrastes e intervalos están bien calculados; se controla la estacionalidad.
Reproducibilidad	Reiniciar y ejecutar todo funciona; se crea o entrega la base; se indican versiones y semilla.
Análisis y comunicación al cliente	La respuesta se entiende sin ser técnico, incluye cifras con su incertidumbre y una recomendación accionable con límites explicados.
IA responsable	Se distingue correlación y causalidad, se explican los límites (datos sintéticos, un solo alojamiento) y se propone una cautela concreta sobre el uso del modelo.

Pasos sugeridos

- **Viernes 30/10:** planning (15 min); el descenso del gradiente y la regresión desde cero se practican en los cuadernos 02 y 03, que son la base del reto.
- **Martes 3/11 (con el cuaderno `S05_07`):** cargad los datos en la base (es la práctica de CRUD), mirad los datos, implementad el gradiente y el modelo con control (festivo y mes), contrastad con `statsmodels` y calculad p-valores e intervalos de confianza.
- **Cierre del martes (40 min):** redactad la respuesta al cliente, revisad con la rúbrica, reiniciad y ejecutad todo, y preparad la demostración.

Guía paso a paso para principiantes

Paso 1 · Cargar los datos en una base de datos (martes 3/11, es la práctica de CRUD, 40 minutos)

1. Abrid la plantilla `S05_07_reto_ocupacion` en Colab (enlace en la ficha del reto).
2. Cargad `ocupacion_diaria.csv` . Debe tener **1096 filas** (tres años de días).
3. Creád `turisdata_reto.db` con una tabla `ocupacion_diaria` , y volved a leerla desde Python con una consulta SQL (`SELECT ... FROM ocupacion_diaria`). Comprobad que siguen siendo 1096 filas.

Paso 2 · Mirar los datos (martes 3/11, 15 minutos)

1. Haced un gráfico de la ocupación a lo largo del tiempo y otro de la **media por mes**.
2. Haced un gráfico de **ocupación frente a temperatura**. ¿Se ve una relación? ¿Hay estaciones que se repiten?
3. **Anotad** lo que veáis (en palabras). Es la base de vuestra respuesta al cliente.

Paso 3 · Regresión lineal desde cero (martes 3/11, 45 minutos)

1. Entended la idea con un ejemplo mínimo: si la ocupación fuera `a + b × temperatura`, ¿qué valores de `a` y `b` dan la mejor recta? El **error** es lo mal que se ajusta; el **gradiente** indica en qué dirección cambiar `a` y `b` para que el error baje.
2. Seguid el cuaderno: programad el **descenso del gradiente** (vosotros, **sin** `LinearRegression`). Haced el **modelo simple** (solo temperatura).
3. Pintad la **curva de aprendizaje**. Debe bajar y estabilizarse. Si «explota» (sube sin parar), la tasa de aprendizaje es demasiado grande: reducidla.
4. Haced el **modelo con control**: añadid *festivo* y *mes* (para controlar la estacionalidad).

Paso 4 · Comparar con una librería (martes 3/11, 15 minutos)

Calculad la misma regresión con `statsmodels` y comparad los coeficientes con los vuestros. **Si coinciden (con una tolerancia razonable), vuestro código es correcto.** Si no, buscad el fallo: la causa más común es una tasa de aprendizaje inadecuada o pocas vueltas del algoritmo.

Paso 5 · Medir la seguridad del resultado (martes 3/11, 15 minutos)

1. Obtened de `statsmodels` el **p-valor** y el **intervalo de confianza** del efecto de la temperatura y del festivo.
2. Calculad el **tamaño del efecto**: cuántos puntos de ocupación cambia con un grado más (o con un festivo).
3. **Interpretad con palabras** cada cifra. Ejemplo (inventado): *«Con un grado más, la ocupación baja unos 0,8 puntos (intervalo de confianza entre 0,5 y 1,1)»*.

Paso 6 · Responder al cliente en 150 palabras (martes 3/11, 15 minutos)

La respuesta tiene que contestar **tres preguntas**:

1. **¿Qué encontráis?** (con cifra e incertidumbre).
2. **¿Qué recomendáis?** (algo que el cliente pueda hacer).
3. **¿Qué no podéis afirmar?** (límites: datos sintéticos, un solo alojamiento, correlación ≠ causalidad).

Contad las palabras. Más de 150, se recorta.

Después: **reiniciad el cuaderno y ejecutadlo todo de arriba abajo** (en Colab: *Entorno de ejecución → Reiniciar y ejecutar todo*). Si falla, corregidlo antes de entregar.

Cómo usar la IA en este reto

Para qué	Ejemplo de petición	Qué tenéis que comprobar
Entender una idea	«Explícame el descenso del gradiente como si fuera bajar una montaña con niebla, sin fórmulas»	Que podáis explicarlo sin mirar
Entender un error de código	«Mi curva de aprendizaje sube en vez de bajar. Este es mi bucle: *[código]*. ¿Qué puede pasar?»	Que probáis la sugerencia y comprobáis que baja
Interpretar un resultado	«Tengo un p-valor de 0,03 y un intervalo de confianza de [−1,1; −0,4]. Explícamelo para un directivo, con un ejemplo»	Que la explicación coincide con **vuestros** números
Revisar la redacción	«Revisa mi respuesta de 150 palabras: ¿hay jerga o algo ambiguo? No cambies las cifras»	Que las cifras siguen siendo las vuestras

Reglas del reto:

- El **descenso del gradiente lo escribís vosotros**. Si lo pide una IA, no podréis explicarlo en la review.
- **Cualquier cifra** que aparezca en la respuesta tiene que salir de vuestro cuaderno.

- Declarad en el cuaderno **dónde habéis usado IA** y **qué habéis comprobado**.

Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
La curva de aprendizaje sube o da NaN	Tasa de aprendizaje demasiado grande o datos sin escalar	Reducid la tasa; escalad la temperatura
Vuestra recta no coincide con `statsmodels`	Pocas vueltas o tasa muy pequeña	Más iteraciones o ajustad la tasa
Decir que «el calor provoca…»	Se confunde correlación con causa	Hablad de «se asocia con» y explicad los límites
Olvidar la estacionalidad	El verano coincide con calor Y con vacaciones	Incluid *mes* en el modelo con control
Dar solo el p-valor	«Es lo que importa»	Dad también el **tamaño** del efecto y el **intervalo**
Cuaderno que solo funciona en un orden raro	Se ejecutaron celdas sueltas	*Reiniciar y ejecutar todo* antes de entregar

Un buen resultado se parece a...

Ejemplo **inventado** de respuesta al cliente (≤ 150 palabras):

> *«Con los datos de tres años, la ocupación baja unos 0,8 puntos por cada grado más de temperatura (intervalo de confianza: 0,5 a 1,1), una vez descontados el mes y los festivos. Los festivos suben la ocupación unos 12 puntos de media. Recomendamos reforzar personal en festivos y no bajar precios por calor sin comprobarlo antes en una prueba controlada. No podemos afirmar que el calor sea la causa: pueden influir otros factores que no están en los datos. Además, los datos son sintéticos y de un solo alojamiento: no deben generalizarse.»*

Fijaos: **cifra con incertidumbre, recomendación concreta, límites explícitos.**

Antes de entregar: autoevaluación rápida

Pregunta	Sí / No
¿Todos los puntos de control (`assert`) del cuaderno pasan?	
¿Existe `turisdata_reto.db` con la tabla `ocupacion_diaria` y 1096 filas?	
¿La regresión desde cero coincide con `statsmodels` y hay curva de aprendizaje?	
¿Hay un p-valor, un intervalo y un tamaño de efecto **interpretados con palabras**?	
¿La respuesta al cliente tiene ≤ 150 palabras, con recomendación y límites?	
¿Los gráficos tienen título y ejes en español?	
¿Funciona *Reiniciar y ejecutar todo* sin errores?	

Si os atascáis

- **No entendemos el gradiente:** repasad `S05_03_gradiente_regresion` con calma; el ejemplo de una sola variable es el mismo que usaréis aquí.
- **La base de datos no se crea:** comprobad que usáis `sqlite3.connect("turisdata_reto.db")` y que habéis hecho `commit` al guardar.
- **No sabemos interpretar el p-valor:** en una frase: *«¿sería raro ver esto si no hubiera efecto?»*. Si sí es raro (p pequeño), hay probable efecto.

- Más ayuda: Cómo se trabaja un reto.

Qué se enseña en la review

Una demostración de 5 minutos: (1) la pregunta y la base de datos, (2) cómo converge vuestro descenso del gradiente y su comparación con la librería, (3) el resultado clave con su intervalo, (4) cómo habéis distinguido lo que los datos muestran de lo que no pueden demostrar (correlación frente a causalidad), (5) la recomendación al cliente.

Definition of Done

- [ ] Todos los puntos de control (`assert`) del cuaderno pasan.
- [ ] `turisdata_reto.db` existe y contiene la tabla `ocupacion_diaria` con 1096 filas.
- [ ] La regresión desde cero coincide con `statsmodels` (tolerancia razonable) y hay una curva de aprendizaje.
- [ ] Hay al menos un p-valor, un intervalo de confianza y una medida del tamaño del efecto interpretados con palabras.
- [ ] La respuesta al cliente (≤ 150 palabras) incluye recomendación y límites.
- [ ] Gráficos con título y ejes en español.
- [ ] Reiniciar y ejecutar todo funciona sin errores; alguien del grupo ha revisado el trabajo con la rúbrica.

Plantilla del cuaderno del reto: Abrir en Colab

Recursos

- Documentación de NumPy (guía para principiantes): https://numpy.org/doc/stable/user/absolute_beginners.html
- Documentación de SciPy Stats: https://docs.scipy.org/doc/scipy/reference/stats.html
- Documentación de `statsmodels` : https://www.statsmodels.org/stable/index.html
- Módulo `sqlite3` de Python: https://docs.python.org/es/3/library/sqlite3.html
- Vídeos «Essence of linear algebra» (3Blue1Brown, con subtítulos): https://www.3blue1brown.com/topics/linear-algebra